

아이지킴이 (AI-Zikimi)

AI 순찰 로봇 & 웹 대시보드 — 프로젝트 기획서 v4 (2인 체제 최종 확정)

"AI가 지켜보는 눈, 사람이 없어도 안심할 수 있는 공간"

본 문서는 최초 기획서(v1)를 기준으로, A담당(웹/AI/데이터) 1차 개발(STEP 1~7, 6.5)을 진행하며 확정된 세부 사양과 설계 변경 사항을 반영한 v2를 거쳐, 하드웨어 배송 지연 및 팀원 C 합류에 따라 역할 분담을 3인 체제로 재구성했다가(v3), 최종적으로 다시 A/B 2인 체제로 확정하고 B의 작업을 4단계로 세분화한 v4 버전입니다.

1. 프로젝트 주제

프로젝트 이름: 아이지킴이 (AI-Zikimi) — "AI"(인공지능) + "지킴이"(지켜주는 존재)의 합성어

한 줄 소개: 사람이 자리를 비운 공간을 로봇카가 주기적으로 순찰하며 촬영하고, Gemini AI가 이상 상황(문 열림, 화재 위험, 낯선 물체, 어질러진 상태 등)을 분석해 웹 대시보드로 실시간 알려주는 무인 순찰 시스템.

해결하려는 문제

- 자취방, 사무실, 반려동물이 있는 집 등 "사람이 없을 때 무슨 일이 일어나는지 모르는" 불안감
- 기존 CCTV는 영상만 보여줄 뿐, "지금 위험한 상황인지 아닌지"를 사람이 직접 판단해야 함
- 고정형 CCTV는 사각지대가 많음

핵심 아이디어

촬영은 로봇이, 판단은 AI가, 확인은 웹에서

2. 전체 서비스 구조

데이터 흐름

ESP32-S3 Sense(촬영) → Wi-Fi → Raspberry Pi(이미지 수신 + 로봇 자율 제어) → HTTP 요청 → Python 서버(FastAPI) → Gemini API(분석) → Supabase(DB+Storage) → Realtime 구독 → 웹 대시보드

구성 요소	역할
ESP32-S3 Sense (카메라)	로봇카 탑재, 주기적(기본 1분) 촬영 및 Wi-Fi 전송
Raspberry Pi	로봇 두뇌. 모터 제어, 이미지 서버 중계, Python 실행
로봇카	체크포인트 순서(3~4개) 기반 완전 자율 순찰, 원격 조작 없음
진동 센서	이동 중 흔들림 측정, 안정 시에만 촬영 (예산 부족 시 초음파 대체)
Python	Pi(제어/수신) + 서버(Gemini 호출, Supabase 저장) 양쪽 핵심 로직
Gemini API	체크리스트 기반 이미지 분석, 위험도 판단
Supabase	DB(4~5개 테이블) + Storage(이미지) + Realtime 구독
웹사이트	로봇 상태·순찰 경로·카메라·분석 결과 실시간 표시 (관찰 전용)

팀 구성 (v4 최종): A(웹/AI/데이터) · B(하드웨어 전체 — 이동/센서/카메라/통신 4단계 세분화) — 총 2인

전원: 유선 연결 상시 전원 (배터리 관리는 구현 범위 제외)

통신 방식: MQTT 등 브로커 없이, Raspberry Pi ↔ FastAPI 서버 간 단순 HTTP POST 채택

3. 순찰 공간

시연 장소: 자취방/원룸 (바닥 평평, 장애물 적음)

체크포인트: 4개 지점 — 현관 → 책상 → 침대 → 주방 (개발 단계에서 실제 좌표/이름 확정)

확장 가능 공간: 반려동물 가정, 소규모 사무실/스터디룸, 창고/매장, 공유숙소 등

4. 주요 기능

기능	설명	구현 상태
자율 순찰 이동	체크포인트 기반 완전 자율 이동, 장애물 회피 후 방향 재정렬	B 담당 (하드웨어 대기)
진동 감지 기반 촬영 안정화	흔들림 측정 후 안정 시 촬영	B 담당 (하드웨어 대기)
AI 이미지 분석	Gemini 체크리스트 기반 상황 설명	완료
위험/이상 상황 판단	문열림/쓰러짐/침입/화재 기준, 안전·주의·위험 3단계	완료
실시간 웹 대시보드	로봇 상태, 최근 사진, 분석 요약, Realtime 자동 갱신	완료
분석 기록 조회	날짜·위험도 다중 필터	완료
위험 알림 기능	위험도 상단 배너 강조(위험=펄스)	완료
이벤트 버스트 촬영 (신규)	위험 감지 시 전/후 추가 촬영으로 상황 변화 추적	완료
순찰 경로 시각화 (신규)	체크포인트 평면도 + 실시간 위치 + 이동 궤적	완료
민감도/알림 설정	판단 기준 강도 조절, 알림 on/off	완료
촬영 속도 자동 조절	진동 수치 기반 이동 속도 조절	시간 여유 시 (보류)

4-1. 이벤트 버스트 촬영 (v2 신규 확정 사양)

당초 기획서에는 없던 기능으로, 개발 과정에서 "위험 감지 시 전후 상황을 사진으로 남기자"는 요구에 따라 신설되었습니다. 실시간 영상 스트리밍 대신, 구현 부담이 적은 연속 스냅샷 방식으로 설계했습니다.

- 트리거: 최초 촬영의 risk_level이 "주의" 또는 "위험"으로 판정될 때
- 후속 촬영: 3장, 촬영 간격 8~10초 (로봇 순찰 흐름에 지장 없는 수준)
- Gemini 재분석: 최초(0번) + 후속 1번째(1번) 사진만 재분석, 나머지 2장은 저장만 하여 API 비용 절감
- 종료 처리: 3장 완료 후 평소 촬영 주기(1분)로 자동 복귀, 종료된 이벤트로 재요청 시 서버가 오류 반환
- 웹 표시: 같은 이벤트로 묶인 사진들을 시간순 슬라이드로 표시 (전/후 스토리보드 형태)

4-2. 순찰 경로 시각화 (v2 신규 확정 사양)

체크포인트에 상대 좌표(원룸 평면도 기준 0~100%)를 부여하여, 웹 대시보드에서 로봇의 현재 위치와 이동 궤적을 지도 형태로 표시합니다.

- 체크포인트 4개 지점을 평면도 위에 고정 표시, 사전 정의된 순찰 순서는 점선으로 안내
- 로봇 현재 위치는 실시간(Realtime) 펄스 링으로 강조
- 실제 이동 이력(movement_logs)이 누적되면 지나온 경로를 실선 궤적으로 표시
- 하드웨어 연결 전에는 궤적 데이터가 비어 있으나, 하드웨어 연결 후 별도 코드 수정 없이 자동으로 채워짐

5. 자율주행 방식 상세

방식: 시간 기반 체크포인트 순찰 (좌표 기반 정밀 복귀는 구현 난이도상 제외)

- 경로를 체크포인트 순서로 미리 정의, 각 구간은 "몇 초 전진 → 몇 도 회전"으로 사전 정의
- 장애물 감지 시 회피 기동 후 원래 방향으로 재정렬 (정밀 좌표 복귀 아님)
- 각 체크포인트 도착 예상 시점에 맞춰 촬영

5-1. 움직이는 장애물 대응 (v4 확정 사양)

정적 장애물과 달리 사람·반려동물처럼 움직이는 장애물은 회피 기동을 반복해도 계속 재감지될 수 있어, 무한 회피 루프에 빠지지 않도록 재시도 횟수와 대기 시간에 명확한 상한을 둡니다.

- 장애물 감지 → 회피 기동 → 재정렬 → 전진을 시도, 재감지 시 동일 절차를 최대 2회까지만 반복
- 2회를 초과하면 억지로 더 피하지 않고 그 자리에서 촬영 — 움직이는 대상의 정체(사람/동물이라면 이미지 분석(stranger_or_animal_detected)으로 자연스럽게 감지되므로, 오히려 시스템 목적에 부합
- 촬영 후 5초 대기했다가 원래 목적지로 재시도 — "촬영 → 5초 대기 → 재시도" 사이클을 최대 2회까지 반복
- 2회 반복 후에도 계속 막히면 더 이상 재시도하지 않고 다음 체크포인트로 스킵하여 순찰을 이어감 (무한 대기 방지)
- 이때도 checkpoint는 원래 향하던 목적지 이름 그대로 전송 — 서버/API 계약 변경 없이 기존 POST /api/v1/images 그대로 사용 가능

6. 위험 판단 기준 상세

Gemini 프롬프트에 아래 체크리스트를 명시하여 일관된 형식(안전/주의/위험)으로 응답을 받습니다.

- 현관문이 열려 있는지
- 바닥에 쓰러진 물체나 어질러진 흔적이 있는지
- 낯선 사람이나 동물이 있는지
- 화재 위험 요소(방치된 전열기구 등)가 있는지

응답 구조: summary(요약) / risk_level(안전·주의·위험) / risk_reason(판단 이유)

민감도 설정 (v2 추가): 웹 설정 화면에서 민감도(낮음/보통/높음)를 조정하면, 서버가 Gemini 프롬프트의 판단 기준 강도를 동적으로 조정합니다. 예: '낮음'은 명백한 위험만 주의로 판단, '높음'은 조금이라도 의심되면 주의로 판단.

분석 실패 시 처리: Gemini 호출 실패(타임아웃, JSON 파싱 오류 등) 시 재시도 없이 즉시 risk_level="주의", summary="분석 실패" 폴백을 저장하여 사용자가 직접 확인하도록 유도합니다. camera_logs와 ai_analysis는 항상 1:1 관계를 유지합니다.

7. 촬영 주기

평상시: 체크포인트 도착 기준(지점 기반) + 기본 1분 간격

위험 감지 시: 이벤트 버스트 촬영(4-1 참고)으로 8~10초 간격 3장 추가 후 평소 주기로 복귀

촬영 주기 자체의 변경(예: 30초→2분)은 하드웨어(B) 영역으로, 웹 설정 화면에서는 제외되었습니다.

8. 웹사이트 UI 구성 (v2 업데이트)

화면	표시 내용	비고
메인 대시보드 (/)	로봇 상태 카드, 위험 알림 개수, 위험 배너, 최신 촬영+분석, 순찰 경로 지도	경로 지도 v2 신규
카메라 화면 (/camera)	촬영 이미지 그리드(위험도별 배지), 클릭 시 원본+분석 결과 모달	-
분석 기록 (/history)	날짜 범위 + 위험도 다중 필터, 리스트/모달	-
설정 화면 (/settings)	판단 민감도, 알림 on/off — 서버(Supabase) 저장, 다른 탭에도 Realtime 반영	촬영주기 항목 제외(v2)

로봇 원격 제어 화면은 완전 자율 방식 채택에 따라 애초부터 제외되었습니다.

9. Gemini API 활용 방법

Gemini 멀티모달(이미지 입력) 기능 사용. 촬영 이미지를 서버가 Gemini에 전달하여 아래 프롬프트로 분석을 요청합니다.

"이 사진은 원룸을 순찰하는 로봇이 촬영한 이미지입니다. 다음 항목을 각각 체크해서 답변해줘: 1) 현관문이 열려 있는지 2) 바닥에 쓰러진 물체나 어질러진 흔적이 있는지 3) 낯선 사람이나 동물이 있는지 4) 화재 위험 요소가 있는지 5) 위 항목 중 하나라도 해당되면 위험도를 [주의] 또는 [위험]으로, 아무 이상 없으면 [안전]으로 분류하고 이유를 설명해줘"

v2에서는 이 프롬프트에 민감도 설정값이 동적으로 삽입되어 판단 강도를 조절합니다.

10. Supabase 데이터베이스 설계 (v2 업데이트)

checkpoints (v2 신규)

컬럼	타입	설명
id	uuid, PK	-
name	text	체크포인트 이름 (예: 현관)
order_index	int, unique	순찰 순서
pos_x, pos_y	numeric	평면도 상대 좌표 (0~100%), 경로 시각화용

robot_status

컬럼	타입	설명
id	uuid, PK	-
state	text	순찰중/정지/이동중/장애물회피중
current_checkpoint_id	uuid, FK	현재 체크포인트 (v2: FK로 변경)
next_checkpoint_id	uuid, FK	다음 목적지 체크포인트 (v2 신규)
last_updated	timestamptz	마지막 상태 갱신 시각

camera_logs

컬럼	타입	설명
id	uuid, PK	-
image_url	text	Supabase Storage 경로 (v2: 한글 대신 checkpoint_id 기반 키 사용)
checkpoint_id	uuid, FK	촬영 체크포인트 (v2: 텍스트→FK 변경)
event_id	uuid, nullable	이벤트 버스트 그룹 ID (v2 신규, 평소 촬영은 null)
sequence_in_event	int, nullable	이벤트 내 순서 0=최초, 1~2=후속 (v2 신규)
captured_at	timestamptz	촬영 시각

ai_analysis

컬럼	타입	설명
id	uuid, PK	-
camera_log_id	uuid, FK	camera_logs 참조

summary	text	AI 요약 설명
risk_level	text	안전/주의/위험
risk_reason	text	위험 판단 이유
created_at	timestampz	분석 시각

movement_logs

컬럼	타입	설명
id	uuid, PK	-
checkpoint_id	uuid, FK	도착 체크포인트 (v2: 텍스트→FK 변경)
obstacle_avoided	boolean	장애물 회피 발생 여부
created_at	timestampz	기록 시각 — 시간순 정렬 시 경로 궤적 재구성

settings (v2 신규)

컬럼	타입	설명
id	uuid, PK	-
sensitivity	text	낮음/보통/높음 (기본값: 보통)
notifications_enabled	boolean	알림 배너 표시 여부

공통: Realtime publication에 5개 테이블 모두 등록, RLS는 읽기 전체 허용 / 쓰기는 서버(service key)만 허용.

11. A ↔ B 인터페이스 — API 계약 (v2 확정)

POST /api/v1/images (multipart/form-data)

필드	타입	필수	설명
file	이미지 파일	예	촬영 사진
checkpoint	텍스트	예	체크포인트 이름 (예: "현관") — uuid 아님, 서버가 내부 변환
event_id	uuid 텍스트	선택	이전 응답값. 평소 촬영 시 생략

응답 (JSON)

필드	설명
event_id	위험 감지 시 새로 발급 (null이면 평소 촬영)
sequence_in_event	0=최초 감지, 1~3=후속
burst_remaining	남은 후속 촬영 수 (0이면 종료)
next_capture_interval_sec	다음 촬영 대기 시간(초). null이면 평소 주기(60초) 복귀

B 담당 로직: 응답에 next_capture_interval_sec이 있으면 그 초만큼 대기 후 같은 event_id로 재호출. null이면 평소 주기로 복귀. 상세 예시/에러 케이스는 별도 docs/api-contract.md 참고.

12. 하드웨어 동작 시나리오 (v2 업데이트)

- Raspberry Pi가 다음 체크포인트를 향해 전진 명령 (시간 기반 이동)

- 장애물 감지 시 회피 기동 후 방향 재정렬
- 체크포인트 도착 예상 시점에서 진동 센서로 안정 상태 확인 후 촬영
- ESP32가 Wi-Fi로 Pi에 이미지 전송, Pi가 FastAPI 서버로 업로드
- 서버가 Gemini 분석 요청 → 결과(summary/risk_level/risk_reason) Supabase 저장
- (v2) risk_level이 "주의" 이상이면 서버 응답에 event_id+next_capture_interval_sec 포함 → B가 8~10초 후 같은 checkpoint에서 같은 event_id로 재촬영, 총 3회 반복 후 평소 주기 복귀
- 웹 대시보드가 Realtime 구독으로 자동 갱신, 위험 시 알림 배너 + 경로 지도에 위치 반영
- 다음 체크포인트를 향해 반복

13. 2인 팀 역할 분담 (v4 최종 — 팀 구성 확정)

경위: 하드웨어 배송 지연으로 잠시 팀원 C의 합류를 논의하여 3인 체제(v3)로 조정했으나, 이후 팀 사정으로 C 없이 진행하기로 최종 확정하여 다시 A/B 2인 체제로 돌아왔습니다. 다만 C가 맡으려던 "카메라/통신" 범위는 B가 흡수하되, 한 사람이 감당할 수 있도록 하위 4단계로 세분화하여 우선순위를 정했습니다.

A 담당 — 웹/AI/데이터 (완료)

- 웹 대시보드(대시보드/카메라/기록/설정) 프론트엔드 + 순찰 경로 시각화 — 완료
- Supabase 테이블 설계 및 연동(Storage 포함), Realtime 구독 — 완료
- Gemini API 연동, 체크리스트 프롬프트 설계 및 민감도 반영 — 완료
- FastAPI 서버(이미지 수신, 이벤트 버스트 로직, 설정 API) — 완료
- B가 하드웨어 없이도 통신 로직을 미리 개발/검증할 수 있도록 서버 외부 터널링(Cloudflare Tunnel) 지원, API 계약서/DB 스키마/테스트용 이미지 공유
- STEP 8 하드웨어 통합 테스트 총괄, 발표/시연 시나리오 준비

B 담당 — 하드웨어 전체 (4단계 세분화)

한 사람이 로봇 이동, 센서, 카메라, 통신을 모두 담당하므로, 아래 순서로 우선순위를 두고 진행합니다.

단계	내용	비고
B-1	로봇 이동 제어 — 체크포인트 기반 자율 이동(전진/회전), 초음파 센서 장애물 회피 및 방향 재정렬	하드웨어 도착 후 최우선
B-2	진동 센서 & 촬영 타이밍 — 흔들림 임계값 실측, 안정 상태일 때만 촬영 신호	하드웨어 도착 후
B-3	ESP32 카메라 촬영/전송 — ESP32-S3 Sense 촬영 코드, Wi-Fi로 Pi에 이미지 전송	하드웨어 도착 후
B-4	Pi ↔ 서버 통신 — API 계약(11장)에 맞춰 file+checkpoint+event_id 전송, 이벤트 버스트 재요청 로직 구현	하드웨어 없이도 A의 서버로 사전 개발/검증 가능, 우선 착수 권장

공통 작업

- API 계약(11장), DB 스키마 기준으로 인터페이스 정합성 유지
- 최종 통합 테스트(로봇 촬영 → AI 분석 → 웹 표시까지 전체 흐름 점검) — 하드웨어 도착 후 A/B 공동 진행

14. 개발 진행 현황 (v2 신규 — 1차 개발 결과)

STEP	내용	상태
1	웹 대시보드 UI/UX (목데이터)	완료
2	Supabase 테이블/SQL 설계	완료

3	Supabase Storage 이미지 업로드	완료
4	Gemini API 이미지 분석	완료
5	FastAPI 이미지 수신 API	완료
6	FastAPI → Gemini → Supabase 전체 연결	완료
6.5	이벤트 버스트 촬영 (추가기능)	완료
7	Supabase Realtime → 웹 자동 갱신	완료
8	B 담당 하드웨어 통합 테스트	하드웨어 준비 대기

여유 작업으로 순찰 경로 시각화, 설정 화면(민감도/알림) 서버 연동까지 추가 완료.

15. 개발 난이도 단계 (v2 반영)

필수 (MVP) — 완료

- ESP32 촬영 + Pi 이미지 수신(HTTP), 로봇카 자율 이동, 장애물 회피(B 담당, 하드웨어 대기)
- Gemini 이미지 분석(체크리스트 기반 요약+위험도) — 완료
- Supabase 저장(camera_logs, ai_analysis) + Storage 업로드 — 완료
- 웹 대시보드(최신 사진+분석 결과 표시) — 완료

추가 기능 — 완료

- 진동 센서 기반 촬영 안정화 (B 담당, 하드웨어 대기)
- 분석 기록 조회/필터 화면 — 완료
- 위험 알림 배너 + 위험 시 추가 촬영(이벤트 버스트) — 완료

시간 여유 시 — 일부 완료

- 체크포인트 이동 경로 시각화 — 완료 (v2)
- 촬영 주기/민감도 설정 화면 — 민감도/알림만 완료, 촬영주기는 하드웨어 영역으로 이관
- 진동 수치에 따른 이동 속도 자동 조절 — 보류 (하드웨어 대기)

16. 최종 요약 (발표용)

항목	내용
프로젝트명	아이지킴이 (AI-Zikimi)
한 줄 소개	AI가 대신 순찰하고 판단하는 무인 공간 모니터링 로봇 시스템
문제 정의	기존 CCTV는 영상만 보여줄 뿐 위험 여부는 사람이 직접 판단, 고정형이라 사각지대 존재
해결 방법	로봇카가 체크포인트 순서로 자율 순찰·촬영, Gemini가 체크리스트 기준으로 즉시 판단, 위험 시에만 알림
핵심 기능	자율 순찰 이동, 진동 감지 촬영 안정화, AI 상황 분석, 위험 알림, 실시간 대시보드, 이벤트 버스트 촬영, 순찰 경로 시각화
사용 기술	ESP32-S3 Sense, Raspberry Pi, Python(FastAPI), Gemini API, Supabase, Next.js
시스템 구조	카메라(촬영) → 로봇/서버(HTTP) → AI(분석) → DB/Storage(저장) → 웹(표시)

차별점	이동형 + AI 판단형 순찰 시스템으로 사각지대 문제와 지속 관찰 부담을 동시에 해결. 체크포인트 기반 구조로 원룸 외 다양한 무인 공간에 손쉽게 확장 가능
-----	---